COMP3220

Fall 2026

Assignment 2

Assignment: Maze Solver with BFS, DFS, and A*

Objective: Implement and visualize BFS, DFS, and A search algorithms to solve a maze using Python's turtle graphics **(or any other graphics library)**.*

📌 Learning Outcomes

- Understand the core concepts behind graph traversal algorithms.
- Implement BFS, DFS, and A* in Python.
- Visualize search processes in a maze environment.
- Develop interactive programs using the turtle module.
- Explain, in writing, why different search strategies behave differently on the same problem.

🗺️ Task Description

Students will:

1. Generate or use a predefined grid-based maze (2D list).
2. Represent the maze visually using the turtle module.
3. Implement BFS, DFS, and A* to find a path from a start to a goal node.
4. Animate the search process and final path using turtle graphics.
5. Run all three algorithms on a fixed benchmark maze (§4) and report the results (§6).

🔧 Requirements

1. Maze Representation

- Use a 2D list (e.g., 1 = wall, 0 = free space, S = start, G = goal).
- Display the maze using turtle with different colors for walls, open spaces, start, and goal.
- Implement path costs for A*.

2. Search Visualization

- As the algorithm explores the maze, color visited nodes.
- Highlight the final path in a distinct color.

3. Algorithm Implementation

- Implement BFS, DFS, and A* as separate functions, following the required interface in §3 exactly.
- Keep your search logic free of any turtle/graphics calls — the functions in §3 must run and return correct results with no window open at all.
- Build your visualization as a separate function/module that takes the path and visited-order returned by §3 and animates them.
- Allow users to switch algorithms (e.g., via input or keypress).
- A possible heuristic function is the Manhattan distance: h(n) = abs(n.x − goal.x) + abs(n.y − goal.y)

Why the split in Algorithm Implementation? Two reasons. First, it's good practice — your search logic shouldn't care how (or whether) it gets drawn. Second, it's what makes §6's comparison table possible to grade quickly: I can run your three functions directly against the benchmark maze without launching a graphics window.

🧪 Required Function Interface

Place the following in a file named maze_solver.py. Your turtle visualization script should import from this file — it should not contain the algorithm logic itself.

```
def bfs(maze, start, goal):
    """
    maze:  2D list of '0'/'1' (or 0/1), same layout as the input maze
    start: (row, col) tuple
    goal:  (row, col) tuple

    Returns a tuple: (path, visited_order, stats)
      path          - list of (row, col) from start to goal, inclusive.
                      Empty list [] if no path exists.
      visited_order - list of (row, col), in the order your algorithm
                      expanded/visited them (used to animate the search).
      stats         - dict with keys:
                      'nodes_expanded' (int), 'path_length' (int),
                      'path_cost' (int or float)
    """

def dfs(maze, start, goal):
    # Same signature and return format as bfs()
    ...

def astar(maze, start, goal, heuristic=manhattan_distance):
    # Same signature and return format as bfs()
    ...
```

Keeping this interface exact is worth 5 points on its own (§7) — it is the single easiest thing in this assignment to get full marks on.

📐 Required Benchmark Maze

Use the exact grid below — unmodified — to produce the comparison table required in §6. You are free to design your own maze(s) (or a maze generator) for your demo video and for any creative work in §5, but the comparison table must come from this maze, so results are directly comparable across submissions.

```
1111111111
1S01001001
1000101101
1110010001
1010000001
1000001101
1000000001
1110000101
10010000G1
1111111111
```

(1 = wall, 0 = open space, S = start at row 1 col 1, G = goal at row 8 col 8, 0-indexed.)

🎮 Creative Enhancements (Required)

You must implement at least one enhancement beyond the base requirements — this is where your own ideas and creativity are actually assessed, so it isn't optional. Pick one or more from the list below, or propose your own (a one-line note in your README is enough to flag it as intentional). Briefly describe what you did and why in your report (§6); I can't give credit for creative work I can't see or read about.

- Real-time switching between search algorithms with keyboard shortcuts.
- Step-by-step animation speed control.
- A custom maze generator (e.g., recursive backtracking) instead of a hardcoded grid.
- An additional algorithm beyond the three required (e.g., Greedy Best-First, Dijkstra, bidirectional BFS).
- A side-by-side "race" view showing two algorithms exploring the same maze at once.
- Your own idea — surprise me. Explain your reasoning in the report and it's eligible.

These 10 points are awarded for genuine extra thought or effort, not visual polish alone — recoloring the maze or changing fonts doesn't qualify on its own.

📦 What to Submit

Submit a single zip file to the VLE containing:

- maze_solver.py — your BFS, DFS, and A* functions, matching §3 exactly.
- Your turtle visualization script(s) — importing from maze_solver.py, not duplicating its logic.
- demo.mp4 or demo.gif — a short screen recording (60–90 seconds is plenty) showing all three algorithms running to completion, including the final path highlight. Your phone camera pointed at your screen is fine if you don't have screen-recording software — just make sure it's readable.
- report.pdf — your Design & Reflection write-up, following §6 exactly.
- README.txt — exact instructions to run your code (Python version, how to launch it, how to switch algorithms).

Submissions missing the demo recording or the report will not be graded on visualization quality or reflection/comprehension marks, regardless of code quality — those criteria are assessed from those two files specifically.

📝 Design & Reflection Report (§6)

One to two pages. Structure it exactly as follows so it's fast to mark — use these as your headings.

A. Maze & Design Choices (≈ 100 words)

Briefly describe your maze source (used the benchmark only, or also built a generator?), your visualization design choices, and anything from §5 you attempted.

B. Comparison Table — Required, on the Benchmark Maze from §4

Algorithm	Path Found?	Path Length (steps)	Nodes Expanded	Path Cost (A* only)
BFS				
DFS				
A*				

C. Reflection (≈ 150–250 words)

In your own words, address:

- Which algorithm performed best on the benchmark maze, and by what measure (path length? nodes expanded?).
- Did DFS find the same path as BFS? Why or why not?
- What was the most interesting or difficult part of building this?

D. Comprehension Questions — Required

Answer each in 2–4 sentences. These are graded for genuine understanding, not length.

1. BFS guarantees the shortest path in terms of number of steps. Does DFS make this same guarantee? Explain why or why not, referencing your results from Part B if helpful.
2. Why is A* generally able to expand fewer nodes than BFS while still finding an optimal path? What property must the heuristic have for this to hold?
3. Describe a maze layout or scenario (not necessarily the benchmark maze) where DFS would perform very poorly compared to BFS and A*.
4. Suppose you added diagonal movement to the maze. What would need to change in your BFS/DFS implementations, and would the Manhattan-distance heuristic still be admissible for A*? Why or why not?

💯 Grading Breakdown (100 points)

Criterion	Points	What Earns Full Marks
BFS correctness	**10**	Auto-tested against the benchmark maze: finds a valid path when one exists; path is contiguous and passes only through open cells.
DFS correctness	**10**	Auto-tested the same way. DFS is not required to find the shortest path — only a valid one.
A* correctness	**10**	Auto-tested the same way, plus: path must be cost-optimal on the benchmark maze (path length 15, matching BFS).
Standard interface compliance	**5**	bfs(), dfs(), and astar() match the required signature exactly, so the test script runs without modification.
Visualization	**10**	Turtle rendering clearly shows walls, open space, start, goal, visited nodes (in order), and the final path in a distinct color.
Demo recording	**10**	One short video/GIF clearly showing all three algorithms running to completion on-screen.
Design & Reflection report	**15**	Comparison table filled in correctly for the benchmark maze; reflection shows genuine engagement, not restated instructions.
Comprehension questions	**10**	Short-answer responses demonstrate real understanding of why the algorithms behave differently — see §6.
Code quality & documentation	**5**	Readable, reasonably commented, sensible naming, logic separated from rendering as required.
Submission compliance	**5**	Correct zip structure, README present and accurate, code runs with no modification needed.
Creative enhancement (required)	**10**	At least one genuine enhancement beyond the base requirements (see §5), clearly described in the report. Polish alone (colors, fonts) does not qualify — it must show extra thought or effort.

The auto-tested items (BFS/DFS/A correctness, interface compliance) are checked by running your maze_solver.py against the benchmark maze and one additional hidden maze of similar size and difficulty — no graphics window required. Everything else is graded from your demo recording and report.*

Submission

Submit your zip file (code + demo recording + report + README) on the VLE by the posted deadline. Late submissions follow the standard course late policy. **The assignment can be done in pairs but is not required. If done in pairs ensure both id numbers are in the code as a comment or in the README.**

If you used an AI tool at any stage (debugging, code review, generating parts of the report), add one line noting that at the end of your README — this isn't penalized when disclosed. Undisclosed, substantial AI-generated work discovered during marking will be treated as an academic integrity matter.